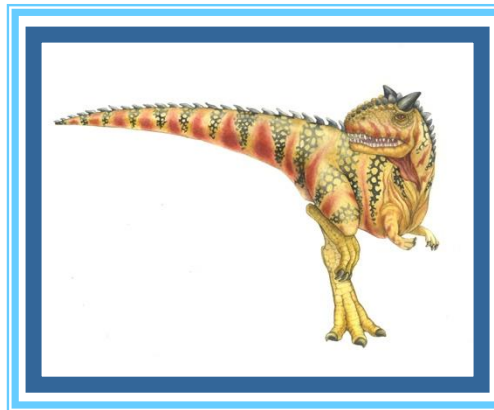


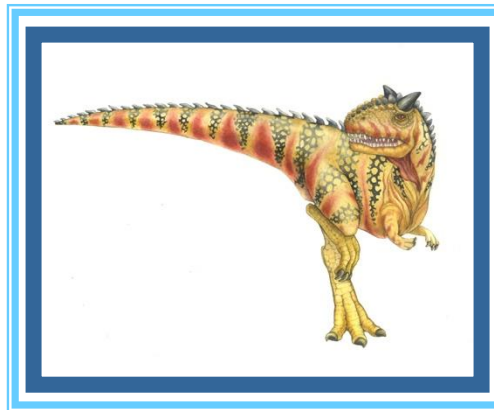
These slides have been modified from the original



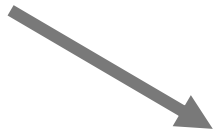
A huge thanks to the
authors of Operating
System Concepts for
providing this resource



These slides have been modified from the original



An indicator on a slide means I've modified something on the slide from the original slides



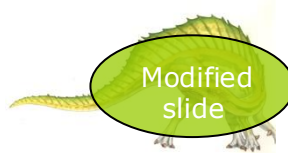
Modified
slide

File Systems



File Concept

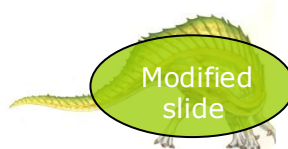
- Logical storage unit
- Types:
 - Data
 - ▶ Numeric
 - ▶ Character
 - ▶ Binary
 - Program
- Contents defined by file's creator
 - Many types
 - ▶ **text file,**
 - ▶ **source file,**
 - ▶ **executable file,**
 - ▶ ...





File Attributes

- **Name** – only information kept in human-readable form
 - **Identifier** – unique tag (number) identifies file within file system
 - **Type** – needed for systems that support different types
 - **Location** – pointer to file location on device
 - **Size** – current file size
 - **Protection** – controls who can do reading, writing, executing
 - **Time, date, and user identification** – data for protection, security, and usage monitoring
-
- Information about files are kept in the directory structure, which is maintained on the disk
 - Many variations, including **extended file attributes** such as file checksum, ...

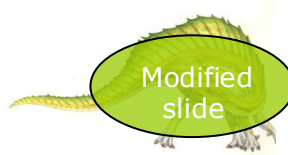




File Operations

- Create, Open, Close
- Write – at **write pointer** location
- Read – at **read pointer** location
- Reposition within file - **seek**
- Delete
- Truncate
- ...

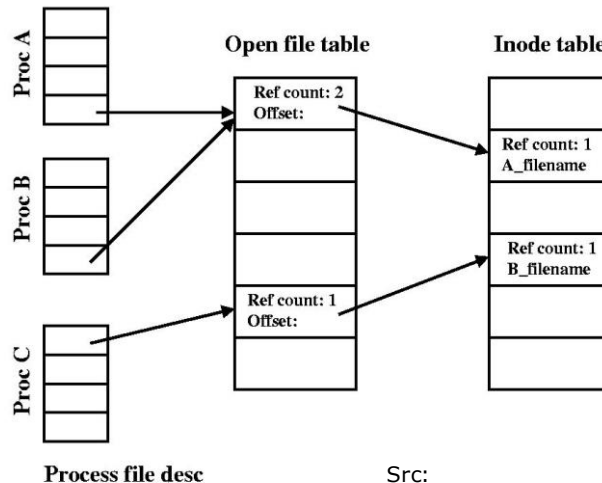
int fseek(FILE *stream, long int offset, int whence) sets the file position of the **stream** to the given **offset**.





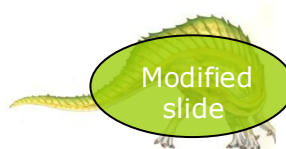
Open Files

- Several pieces of data are needed to manage open files:
 - **Open-file table**: tracks open files
 - File pointer: pointer to last read/write location, per process that has the file open
 - **File-open count**: counter of number of times a file is open – to allow removal of data from open-file table when last processes closes it
 - Disk location of the file: cache of data access information
 - Access rights: per-process access mode information
 - ...



Starting point (for the curious):
<https://github.com/torvalds/linux/blob/master/include/linux/fdtable.h>

Src:
<https://sites.cs.ucsb.edu/~rich/class/cs170/notes/FileSystem/filetable.rich.jpg>

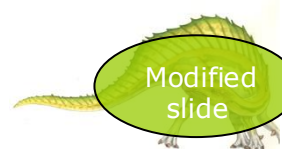




File Types – Name, Extension

file type	usual extension	function
executable	exe, com, bin or none	ready-to-run machine-language program
object	obj, o	compiled, machine language, not linked
source code	c, cc, java, pas, asm, a	source code in various languages
batch	bat, sh	commands to the command interpreter
text	txt, doc	textual data, documents
word processor	wp, tex, rtf, doc	various word-processor formats
library	lib, a, so, dll	libraries of routines for programmers
print or view	ps, pdf, jpg	ASCII or binary file in a format for printing or viewing
archive	arc, zip, tar	related files grouped into one file, sometimes compressed, for archiving or storage
multimedia	mpeg, mov, rm, mp3, avi	binary file containing audio or A/V information

On Linux we make a file executable with:
`chmod u+x filename`
(for the user)

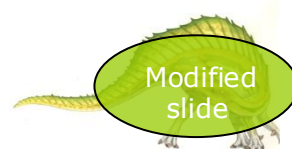




Direct Access (/Relative Access)

- Operations
 - `read  $n$`
 - `write  $n$`
 - `position to  $n$`
 - ▶ `read next`
 - ▶ `write next`
 - ▶ `rewrite  $n$`
- n = **relative block number**

Can read and write out of order...
This is what we are most familiar with.

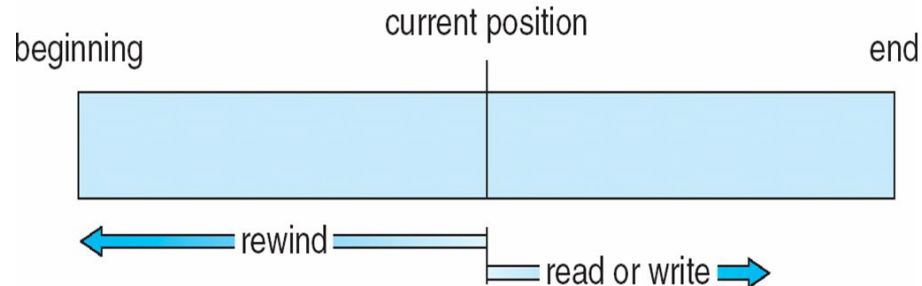




Sequential Access for files

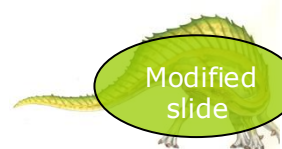
- Operations
 - read next
 - write next
 - Reset

- Figure



<https://spectrum.ieee.org/why-the-future-of-data-storage-is-still-magnetic-tape>
<https://research.ibm.com/blog/tape-density-record>

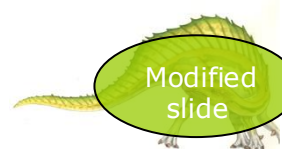
...





Simulation of Sequential Access on Direct-access File

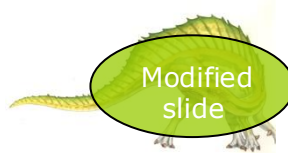
sequential access	implementation for direct access
<i>reset</i>	<i>cp = 0;</i>
<i>read next</i>	<i>read cp;</i> <i>cp = cp + 1;</i>
<i>write next</i>	<i>write cp;</i> <i>cp = cp + 1;</i>





Operations Performed on Directories

- Search for a file
- Create a file
- Delete a file
- List a directory
- Rename a file
- (conveniently) Traverse the file system
- ...

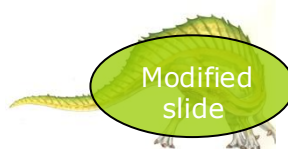




Directory Organization

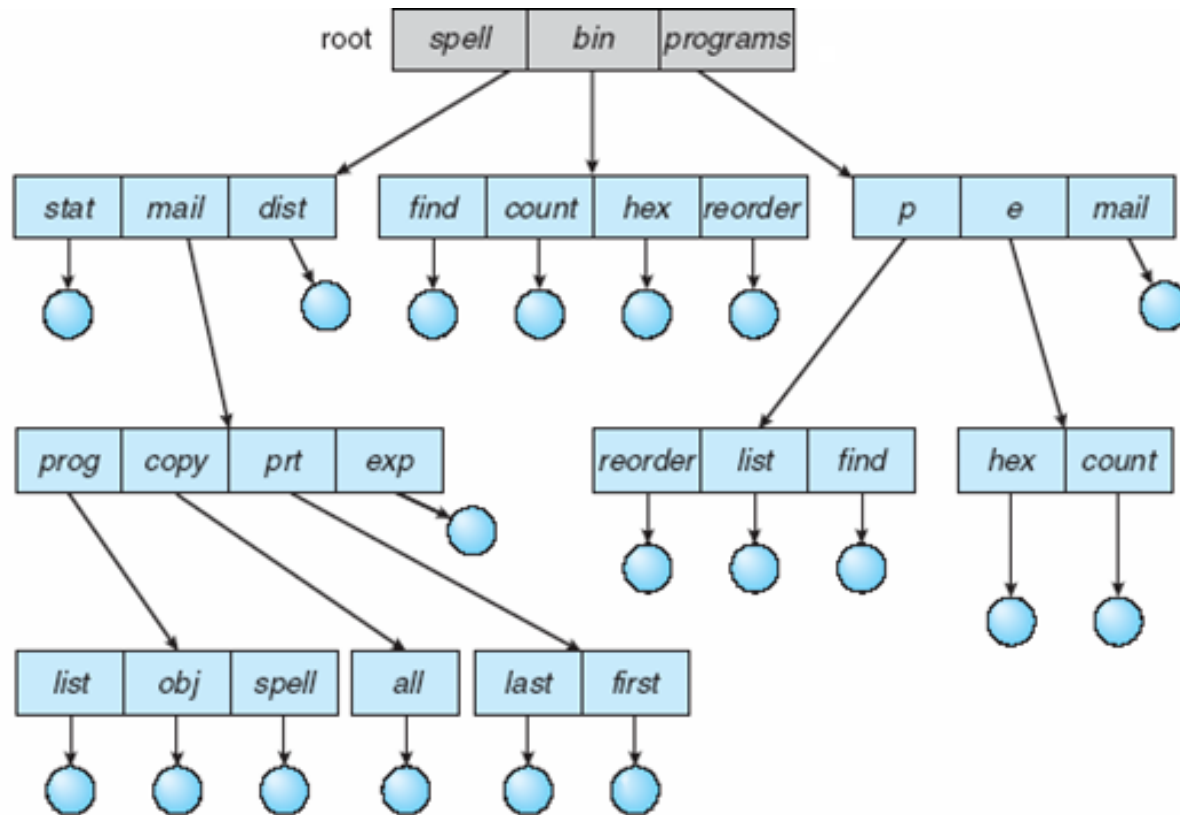
The directory is organized logically to enable:

- Efficiency – locating a file quickly
- Naming – convenient to users e.g.
 - Two users can have same name for different files
 - The same file can have several different names (**linking**)
- Grouping – logical grouping of files by properties, (e.g., all Java programs, all games, ...)





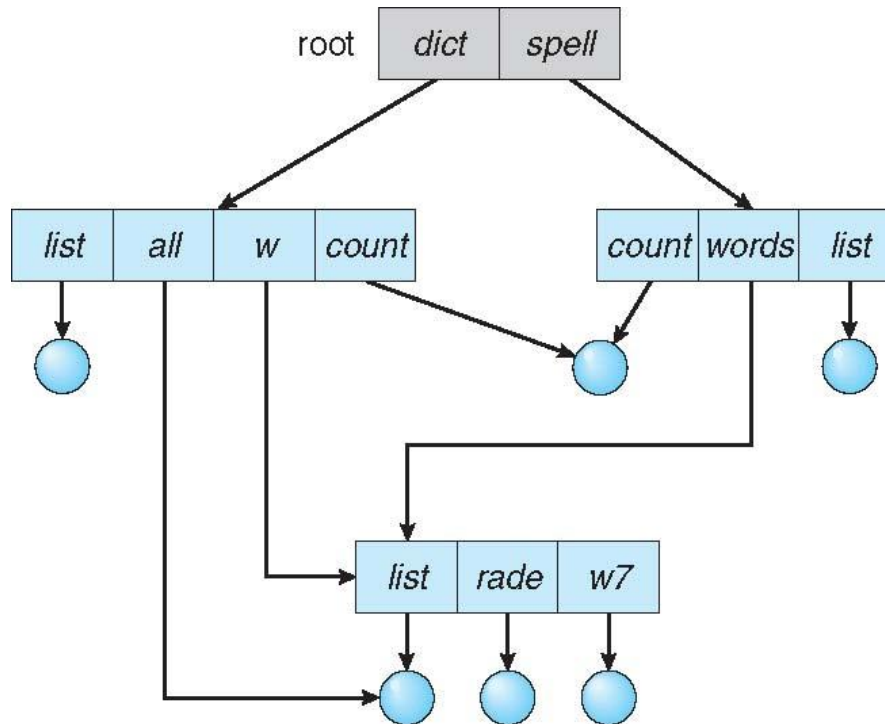
Tree-Structured Directories





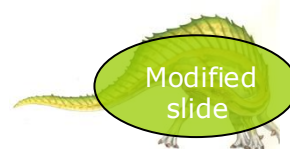
Acyclic-Graph Directories

- Have **shared** subdirectories and files
- Example



In (link) command
in linux/unix

See:
man ln



Soft vs hard link?

Hard links ...

```
(base) ham@hammerGPU:~/links$ mkdir dir1 dir2
(base) ham@hammerGPU:~/links$ echo "Alloy" >> dir1/myfile
(base) ham@hammerGPU:~/links$ ln dir1/myfile dir2/hardlink
(base) ham@hammerGPU:~/links$ ls -li dir1/myfile
24573321 -rw-rw-r-- 2 ham ham 6 Mar 10 00:19 dir1/myfile
(base) ham@hammerGPU:~/links$ ls -li dir2/hardlink
24573321 -rw-rw-r-- 2 ham ham 6 Mar 10 00:19 dir2/hardlink
(base) ham@hammerGPU:~/links$
```


Soft vs hard link?

Data structure for a file system object (file/dir), that store attributes, location, etc

Hard links point to the same inode

```
(base) ham@hammerGPU:~/links$ mkdir dir1 dir2
(base) ham@hammerGPU:~/links$ echo "Alloy" >> dir1/myfile
(base) ham@hammerGPU:~/links$ ln dir1/myfile dir2/hardlink
(base) ham@hammerGPU:~/links$ ls -li dir1/myfile
24573321 -rw-rw-r-- 2 ham ham 6 Mar 10 00:19 dir1/myfile
(base) ham@hammerGPU:~/links$ ls -li dir2/hardlink
24573321 -rw-rw-r-- 2 ham ham 6 Mar 10 00:19 dir2/hardlink
(base) ham@hammerGPU:~/links$
```

ls -i
ln

The '-i' gives us inode information
The link command (see man ln)

The **inode** is a data structure in a Unix-style file system that describes a file-system object such as a file or a directory. Each inode stores the attributes and disk block locations of the object's data.

Soft vs hard link?

That means if I make a change to dir1/myfile, and look in dir2/hardlink I'll see the change, and vice versa!

```
(base) ham@hammerGPU:~/links$ echo "ALOY" > dir1/myfile
(base) ham@hammerGPU:~/links$ cat dir1/myfile
ALOY
(base) ham@hammerGPU:~/links$ cat dir2/hardlink
ALOY
(base) ham@hammerGPU:~/links$ echo "YOLA" > dir2/hardlink
(base) ham@hammerGPU:~/links$ cat dir1/myfile
YOLA
(base) ham@hammerGPU:~/links$
```

Soft vs hard link?

And if I delete the *original* file?

Since the (hard) link points to the inode, the link (file) remains.

```
(base) ham@hammerGPU:~/links$ rm dir1/myfile
(base) ham@hammerGPU:~/links$ ls -li dir2/hardlink
24573321 -rw-rw-r-- 1 ham ham 5 Mar 10 00:24 dir2/hardlink
(base) ham@hammerGPU:~/links$ cat dir2/hardlink
YOLA
(base) ham@hammerGPU:~/links$
```

Soft vs hard link?

What would happen if I created a new file with the same name dir1/myfile? E.g., **echo "t" > dir1/myfile**

They are different inodes!

```
(base) ham@hammerGPU:~/links$ ls -li dir1/myfile
24573326 -rw-rw-r-- 1 ham ham 4 Mar 10 00:28 dir1/myfile
(base) ham@hammerGPU:~/links$ ls -li dir2/hardlink
24573321 -rw-rw-r-- 1 ham ham 5 Mar 10 00:24 dir2/hardlink
```

Soft vs hard link?

What would happen if I created a new file with the same name dir1/myfile? E.g., **echo "t" > dir1/myfile**

They are different inodes!

```
(base) ham@hammerGPU:~/links$ ls -li dir1/myfile
24573326 -rw-rw-r-- 1 ham ham 4 Mar 10 00:28 dir1/myfile
(base) ham@hammerGPU:~/links$ ls -li dir2/hardlink
24573321 -rw-rw-r-- 1 ham ham 5 Mar 10 00:24 dir2/hardlink
```

Test for yourself, write something into dir1/myfile and see if it's the same in dir2/hardlink

Soft vs hard link?

Soft links behave similarly, but with some important differences...

They don't point to the same inode on the disk, instead they point to the file



```
(base) ham@hammerGPU:~/links$ mkdir dir1 dir2
(base) ham@hammerGPU:~/links$ echo "RAILROAD" > dir1/myfile
(base) ham@hammerGPU:~/links$ ln -s dir1/myfile dir2/softlink
(base) ham@hammerGPU:~/links$ ls -li dir1/myfile
24573321 -rw-rw-r-- 1 ham ham 9 Mar 10 00:30 dir1/myfile
(base) ham@hammerGPU:~/links$ ls -li dir2/softlink
24573326 lrwxrwxrwx 1 ham ham 11 Mar 10 00:30 dir2/softlink -> dir1/myfile
```


Soft vs hard link?

So if I delete the original file... it can't access it!

```
(base) ham@hammerGPU:~/links$ rm dir1/myfile
(base) ham@hammerGPU:~/links$ cat dir2/softlink
cat: dir2/softlink: No such file or directory
(base) ham@hammerGPU:~/links$ ls -li dir2/softlink
24573326 lrwxrwxrwx 1 ham ham 11 Mar 10 00:30 dir2/softlink -> dir1/myfile
(base) ham@hammerGPU:~/links$
```

Soft vs hard link for directories?

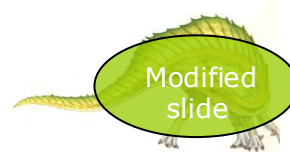
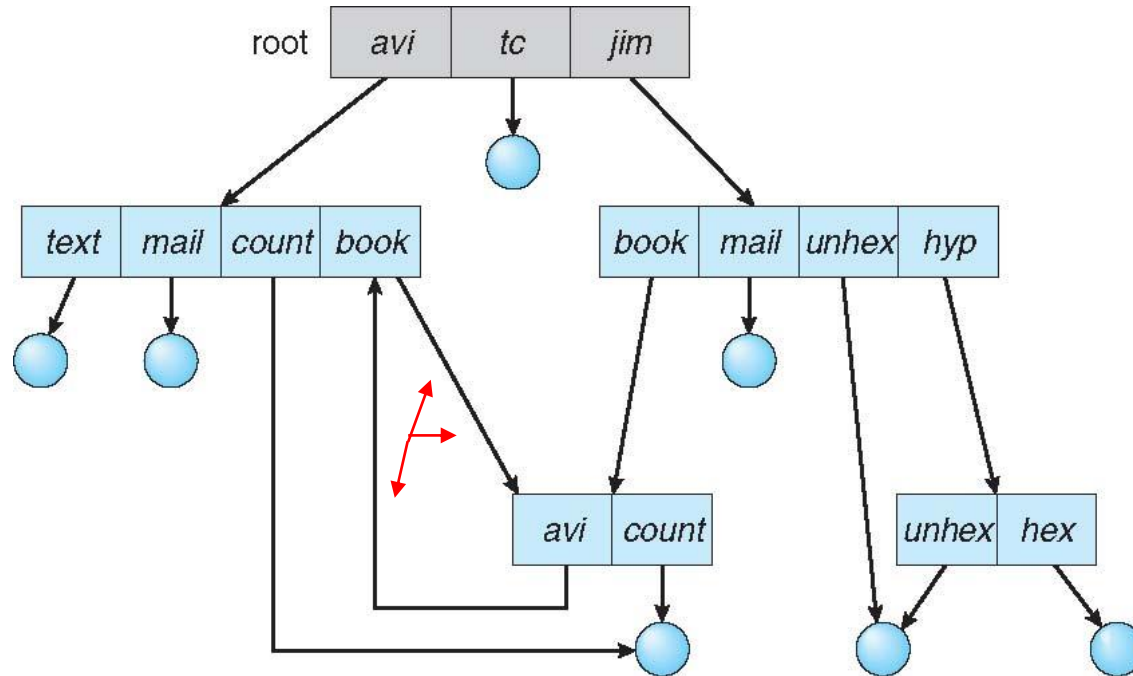
Why wouldn't we want to be able to create a hard link for a directory?

```
(base) ham@hammerGPU:~/links$ mkdir dir1
(base) ham@hammerGPU:~/links$ touch dir1/file1
(base) ham@hammerGPU:~/links$ touch dir1/file2
(base) ham@hammerGPU:~/links$ ln dir1 dir2
ln: dir1: hard link not allowed for directory
(base) ham@hammerGPU:~/links$ ln -s dir1 dir2
(base) ham@hammerGPU:~/links$ ls dir1/*
dir1/file1  dir1/file2
(base) ham@hammerGPU:~/links$ ls dir2/*
dir2/file1  dir2/file2
```

Test all of this for yourself!



General Graph Directory

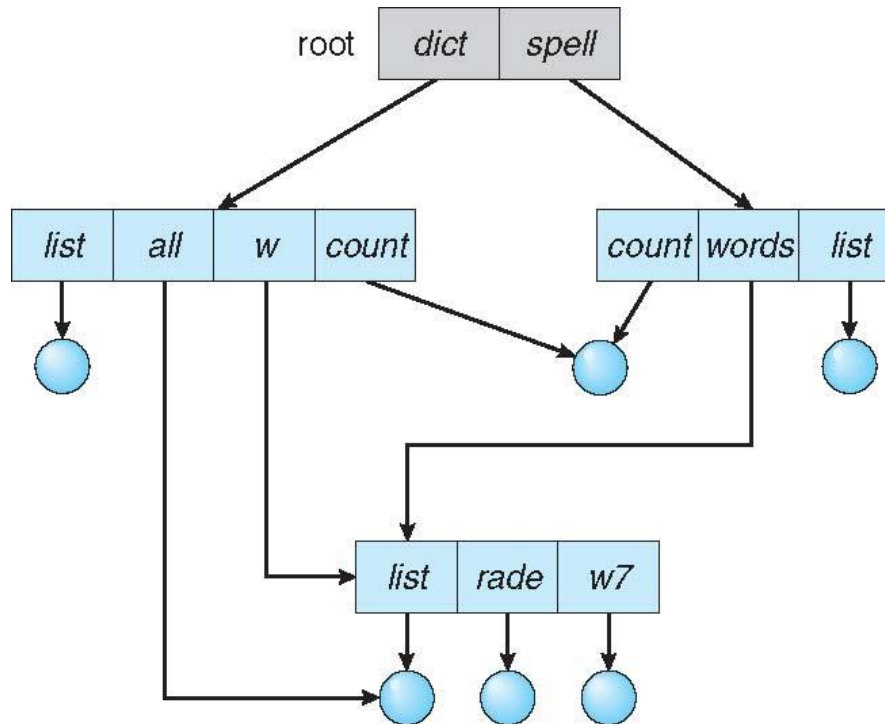


Modified
slide

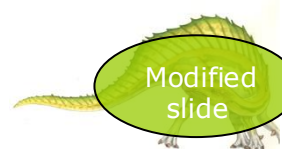


Acyclic-Graph Directories

- Have shared subdirectories and files
- Example



With acyclic graph directories (no hard links), we avoid cycles that may cause issue e.g. infinite loops in file system traversal

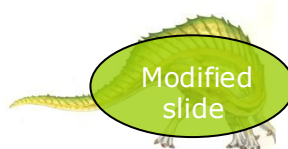




Protection

- File owner/creator should be able to control:
 - What can be done
 - By whom

- Types of access
 - **Read**
 - **Write**
 - **Execute**
 - **Append**
 - **Delete**
 - **List**



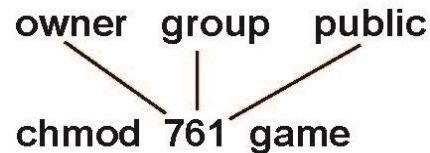


Access Lists and Groups in Unix

- Mode of access: read, write, execute
- Three classes of users on Unix / Linux

			RWX
a) owner access	7	⇒	1 1 1
			RWX
b) group access	6	⇒	1 1 0
			RWX
c) public access	1	⇒	0 0 1

- Ask manager to create a group (unique name), say G, and add some users to the group.
- For a file (say *game*) or subdirectory, define an appropriate access.



- Attach a group to a file
chgrp **G** **game**

Example: chmod 761 filename





A Sample UNIX Directory Listing

ls -lh

-rw-rw-r--	1 pb	g staff	31200	Sep 3 08:30	intro.ps
drwx-----	5 pb	g staff	512	Jul 8 09:33	private/
drwxrwxr-x	2 pb	g staff	512	Jul 8 09:35	doc/
drwxrwx---	2 pb	g student	512	Aug 3 14:13	student-proj/
-rw-r--r--	1 pb	g staff	9423	Feb 24 2003	program.c
-rwxr-xr-x	1 pb	g staff	20471	Feb 24 2003	program
drwx--x--x	4 pb	g faculty	512	Jul 31 10:31	lib/
drwx-----	3 pb	g staff	1024	Aug 29 06:52	mail/
drwxrwxrwx	3 pb	g staff	512	Jul 8 09:35	test/

chgrp groupname file_or_folder
chown owner:group file_or_folder
chown owner file_or_folder

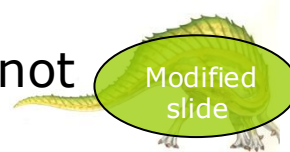
chmod u+x file_or_folder
chmod u-x file_or_folder
chmod o-w file_or_folder
chmod o+r file_or_folder
... (see "man chmod")

Executable for user

Non executable for user

Not writeable by those
not the user or group

Readable by those not
the user or group



Permissions

```
(base) ham@hammerGPU:~/links/perms$ touch t1 t2 t3
(base) ham@hammerGPU:~/links/perms$ chmod u+x t1
(base) ham@hammerGPU:~/links/perms$ ls -alh
total 8.0K
drwxrwxr-x 2 ham ham 4.0K Mar 10 00:59 .
drwxrwxr-x 3 ham ham 4.0K Mar 10 00:56 ..
-rwxrw-r-- 1 ham ham  0 Mar 10 00:59 t1
-rw-rw-r-- 1 ham ham  0 Mar 10 00:59 t2
-rw-rw-r-- 1 ham ham  0 Mar 10 00:59 t3
(base) ham@hammerGPU:~/links/perms$
```

Test all
of this
for
yourself!

Where is this user information stored?

- In /etc/group and /etc/passwd

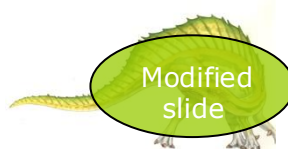
```
(base) ham@hammerGPU:~/links/perms$ sudo cat /etc/group
[sudo] password for ham:
root:x:0:
daemon:x:1:
bin:x:2:
sys:x:3:
adm:x:4:syslog,ham
tty:x:5:syslog
disk:x:6:
lp:x:7:
mail:x:8:
news:x:9:
uucp:x:10:
man:x:12:
proxy:x:13:
kmem:x:15:
dialout:x:20:
```

```
ham:x:1000:
```



Disk Structure

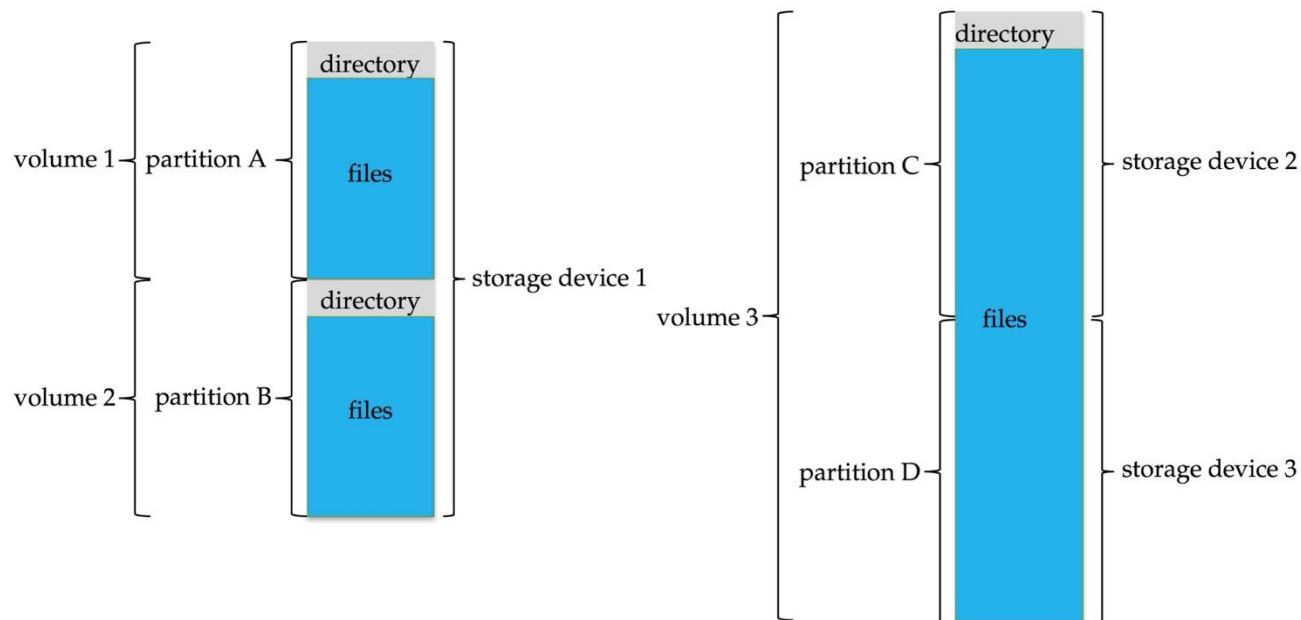
- Disk can be subdivided into **partitions**
- We can even... Disks or partitions can be **RAID** protected against failure
- Disk or partition can be used **raw** – without a file system, or **formatted** with a **file system** (we are interested in the latter)
- Entity containing file system is known as a **volume**
 - Could be in a single partition, or spread across multiple disks (like with RAID)
- In addition to **general-purpose file systems** there are many **special-purpose file systems**, frequently all within the same operating system or computer



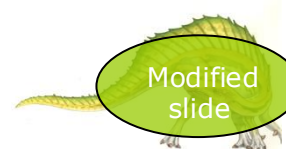


File System

- General-purpose computers can have multiple storage devices
- Devices can be *sliced* into partitions, which hold volumes
- Volumes can also span multiple partitions/drives
- Each volume usually formatted into a file system
- # of file systems varies, typically dozens available to choose from
- Typical storage device organization:



Look at
LVM ...
if
curious



The filesystem on my Linux box

Disk: /dev/nvme0n1					
Size: 465.78 GiB, 500107862016 bytes, 976773168 sectors					
Label: gpt, identifier: 70649795-090B-4772-9DEB-F82C16A613B6					
Device	Start	End	Sectors	Size	Type
/dev/nvme0n1p1	2048	1050623	1048576	512M	EFI System
>> /dev/nvme0n1p2	1050624	910340095	909289472	433.6G	Linux filesystem
/dev/nvme0n1p3	910340096	976771071	66430976	31.7G	Linux swap
Free space	976771072	976773134	2063	1M	

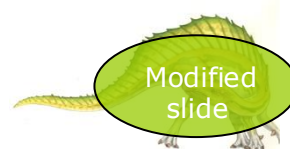


Partitions and Mounting

- Partition can be a volume containing a file system (“cooked”) or **raw** – just a sequence of blocks with no file system
 - ... it’s going to be *formatted* to some filesystem type
- **Root partition** contains the OS, other partitions can hold other OSes, other file systems, or be raw
 - Mounted at boot time
 - Other partitions can mount automatically or manually on **mount points** – location at which they can be accessed
- At mount time, file system consistency checked (**fsck**)
 - Is all metadata correct?
 - ▶ If not, fix it, try again
 - ▶ If yes, add to mount table, allow access



We get to this in a moment

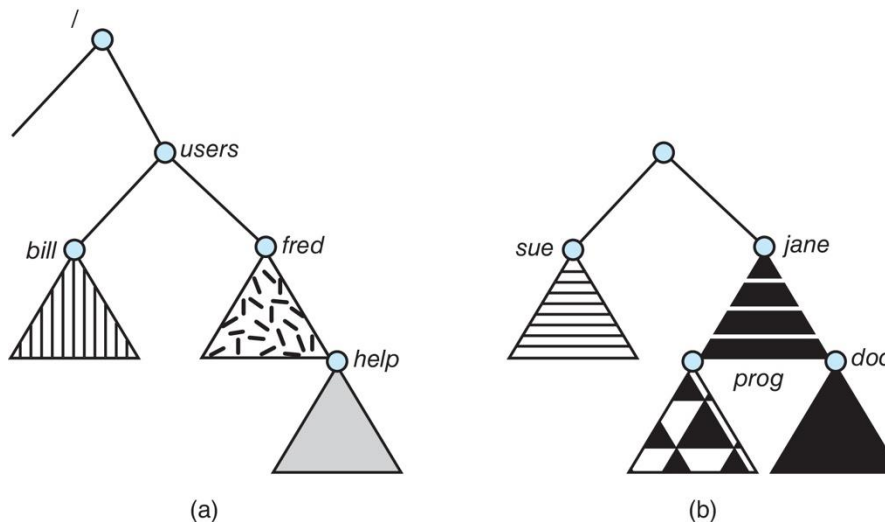




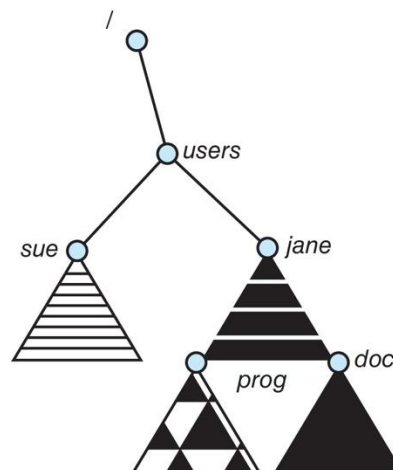
File Systems and Mounting

(a) Unix-like file system directory tree

(b) Unmounted file system



After mounting (b) into the existing directory tree





Types of File Systems

- Many general-purpose file systems (ext4, btrfs, ntfs, zfs, ReFS, exFAT, ...)
- But systems frequently have many file systems, some general- and some special- purpose
- Consider Linux has
 - tmpfs – memory-based volatile FS for fast, temporary I/O
 - udev – exposes underlying hardware as a file (through kernel)
 - procfs – kernel interface to process structures
 - Ext4, reiserfs, ... – general purpose file systems

https://en.wikipedia.org/wiki/Comparison_of_file_systems

Filesystem	Size	Used	Avail	Use%	Mounted on
udev	16G	0	16G	0%	/dev
tmpfs	3.2G	2.5M	3.2G	1%	/run
/dev/nvme0n1p2	427G	387G	19G	96%	/
tmpfs	16G	4.0K	16G	1%	/dev/shm





The Linux Proc File System

root@hammerGPU:~# ls /proc/

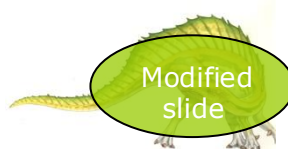
1	1109	1174	1480	1978	28	326	3683	3874492	41	5789	81	driver	scsi
10	1110	1183	15	2	29	3260	3704352	3874493	412	58	82	execdomains	self
1046	1111	12	151	20	3	327	3726	3874495	413	585	83	fb	slabinfo
1047	1112	13	152	2030	30	328	3768740	3874496	414	59	84	filesystems	softirqs
1050	1113	1330	154	2032	302	329	3775381	3874497	416	590	86	fs	stat
1051	1114	1331	155	2070	303	33	3776	3874501	417	591	87	interrupts	swaps
1056	1115	1332	157	21	304	330	3781854	3874502	42	592	88	iomem	sys
1057	1116	1333	1583	2199	305	3338	3797859	3874526	432	593	89	ioports	sysrq-trigger
1061	1117	1334	159061	22	3076	34	3798382	3876051	433	6	9	irq	sysvipc
1063	1118	1335	16	2217	3078	3440	38	3880212	435	60	90	kallsyms	thread-self
1066	1119	1336	1614	2223	308	3443	3805030	3881194	44	60894	90070	kcore	timer_list
1072	1120	1337	167	22459	3089	3446	3811348	3881195	45	62	91	keys	tty
1073	1122	1338	17	2250	309	3449	3817531	3885634	46	63	910	key-users	uptime
1077	1123	1339	1738	2259	3092	3452	3820	3885684	47	64	91205	kmsg	version
1080	1124	1340	1745	23	3093	3458	3831187	3885685	48	647	919	kpagecgroup	version_signature
1082	1125	1341	176	2386	3094	3461	3833	3885721	492	648	920	kpagecount	vmallocinfo
1083	1126	1342	179	24	3116	3464	3838	3885726	50	65	921	kpageflags	vmstat
1084	1127	138	18	2436	313	3467	3846614	3885733	51	66	943518	loadavg	zoneinfo
1086	1128	139	1800	2439	314	3470	3850306	3885744	515	68	992	locks	
1087	1129	14	1911	2455	315	3473	3851482	3885870	518	69	acpi	mdstat	
1088	1130	140	1912	2534	316	3476	3856306	3885930	52	70	asound	meminfo	
1089	1131	141	1913	2542	317	35	3861158	3886188	523	71	buddyinfo	misc	
1090	1133	1411	1914	2549	318	3509	3869853	3886831	529	72	bus	modules	
1091	1134	1413598	1915	2560	319	3547	3870829	3886835	5290	74	cgroups	mounts	
1092	1136	142	1916	26	32	357	3872670	3887096	53	740	cmdline	mtrr	
1096	1140	1428	1917	2605	320	3598	3872671	3887097	531	75	consoles	net	
1097	1144	143	1922	263	321	3599	3872679	39	5374	76	cpuinfo	pagetypeinfo	
11	1146	1434	1946	264	322	36	3872719	3949	54	77	crypto	partitions	
1104	1147	144	1947	265	323	3631	3872933	3994	56	770	devices	pressure	
1107	1152	145	1961	267	324	3639858	3872976	4	569	78	diskstats	sched_debug	
1108	1165	146	197	27	325	3649047	3872977	40	57	80	dma	schedstat	





The Linux Proc File System

- The **proc file system** does not store data, rather, its contents are computed on demand according to user file I/O requests
- **proc** implements a directory structure, with file contents within...
 - When data is read from one of these files, **proc** collects the appropriate information, formats it into text form and places it into the requesting process' s read buffer
- We generally don't use /proc directly (we can, but we don't). We interface with it using the sysctl command
 - E.g. **sysctl net.ipv4.ip_forward net.ipv4.ip_forward = 0**



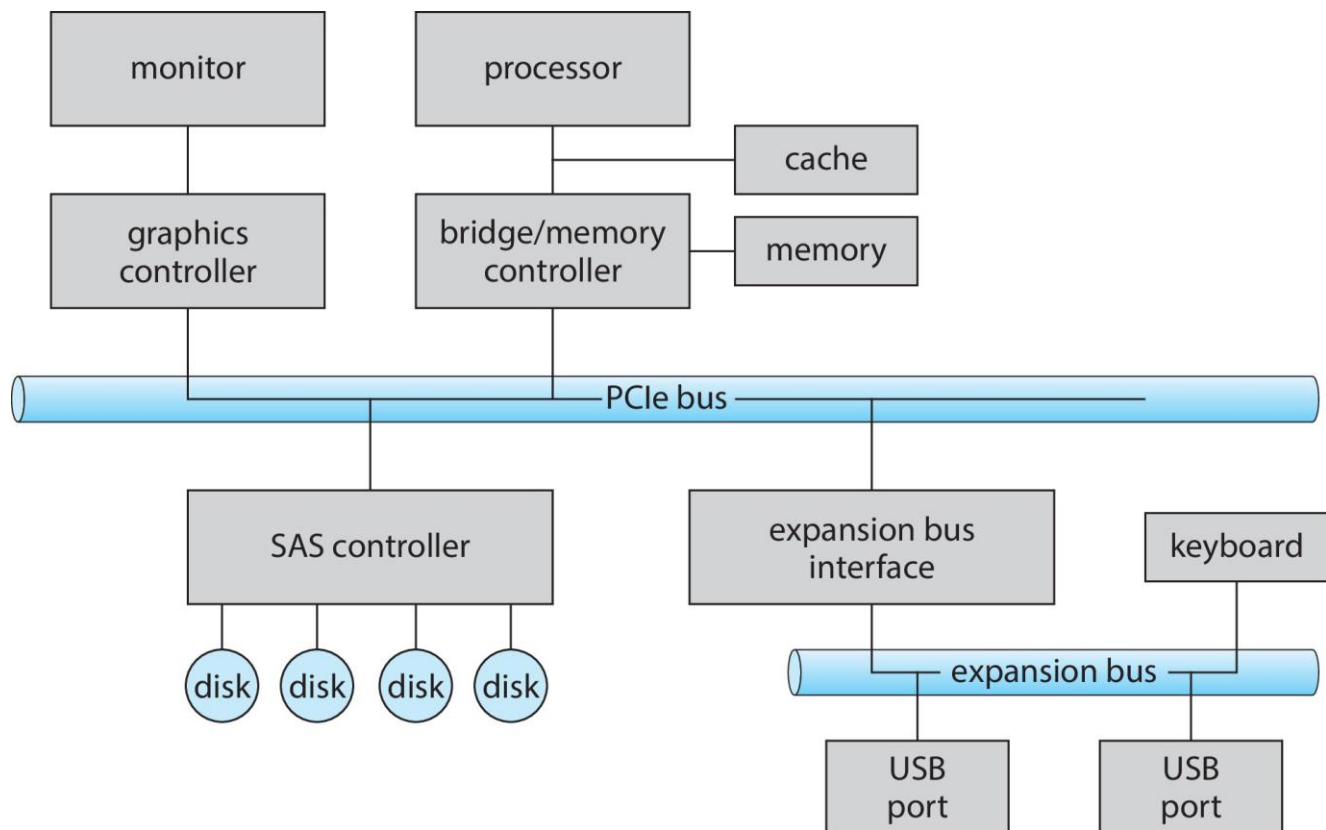


I / O Input / Output





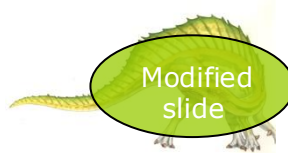
A Typical PC Bus Structure





I/O Hardware

- Incredible variety of I/O devices
 - Storage
 - Transmission
 - Human-interface
 - ...
- Common concept is... – signals from I/O devices interface with computer
 - **Port** – connection point for device
 - **Bus - daisy chain** or shared direct access
 - ▶ **PCI** bus common in PCs and servers, PCI Express (**PCIe**)
 - ▶ **expansion bus** connects relatively slow devices
 - ▶ ...





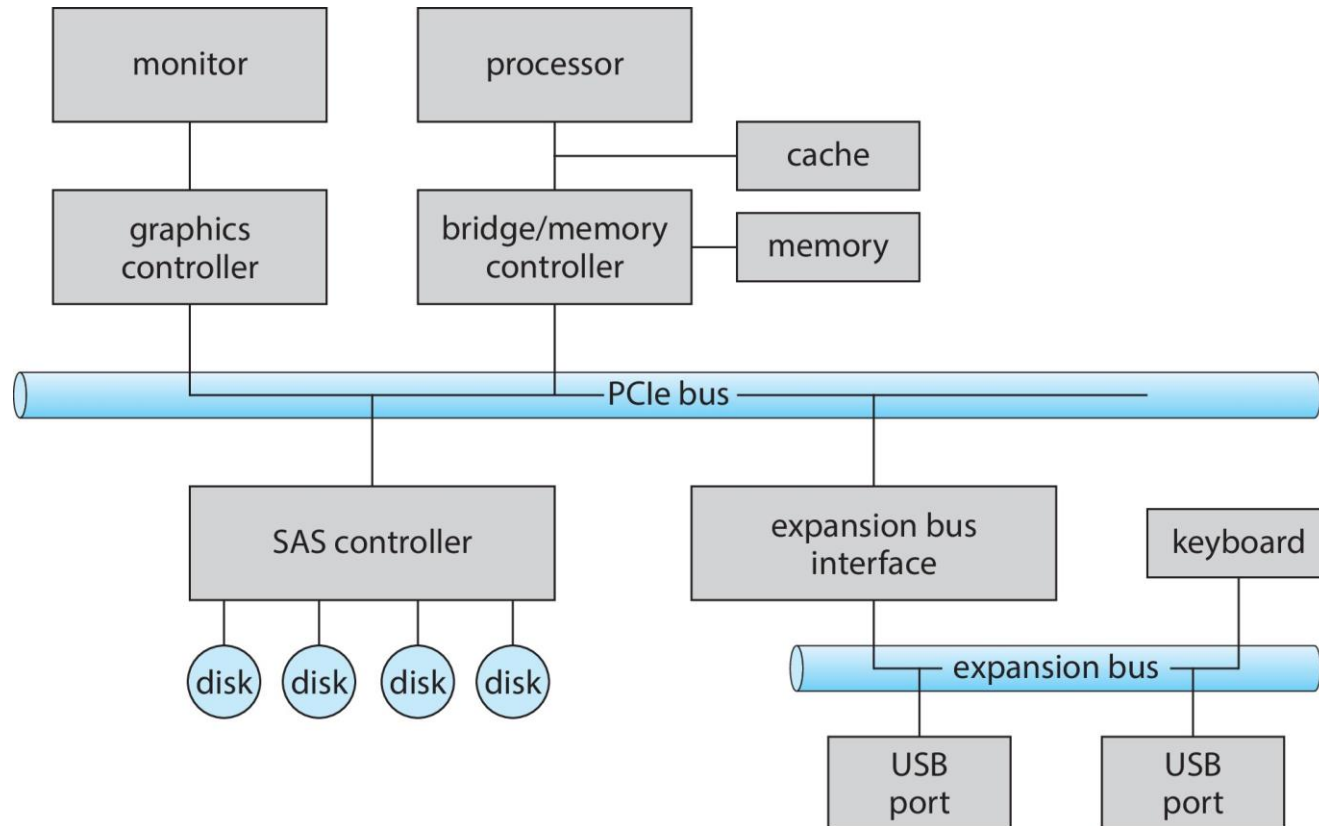
I/O Hardware (Cont.)

- **Controller (host adapter)** – electronics that operate port, bus, device
 - ▶ Contains processor, microcode, private memory, bus controller, etc.
 - Some talk to per-device controller with bus controller, microcode, memory, etc.





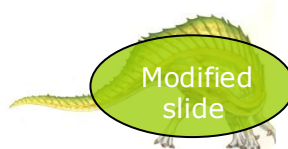
A Typical PC Bus Structure





I/O Hardware (Cont.)

- I/O instructions control devices
- Devices usually have registers where device driver places commands, addresses, and data to write, or read data from registers after command execution
 - Data-in register, data-out register, status register, control register
 - Typically 1-4 bytes, or FIFO buffer
- Devices have addresses, used by
 - Direct I/O instructions
 - **Memory-mapped I/O**
 - ▶ Device data and command registers mapped to processor address space
 - ▶ Especially for large address spaces (graphics)





Device I/O Port Locations on PCs (partial)

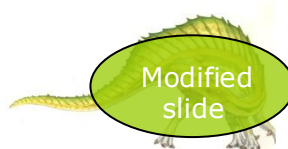
I/O address range (hexadecimal)	device
000–00F	DMA controller
020–021	interrupt controller
040–043	timer
200–20F	game controller
2F8–2FF	serial port (secondary)
320–32F	hard-disk controller
378–37F	parallel port
3D0–3DF	graphics controller
3F0–3F7	diskette-drive controller
3F8–3FF	serial port (primary)





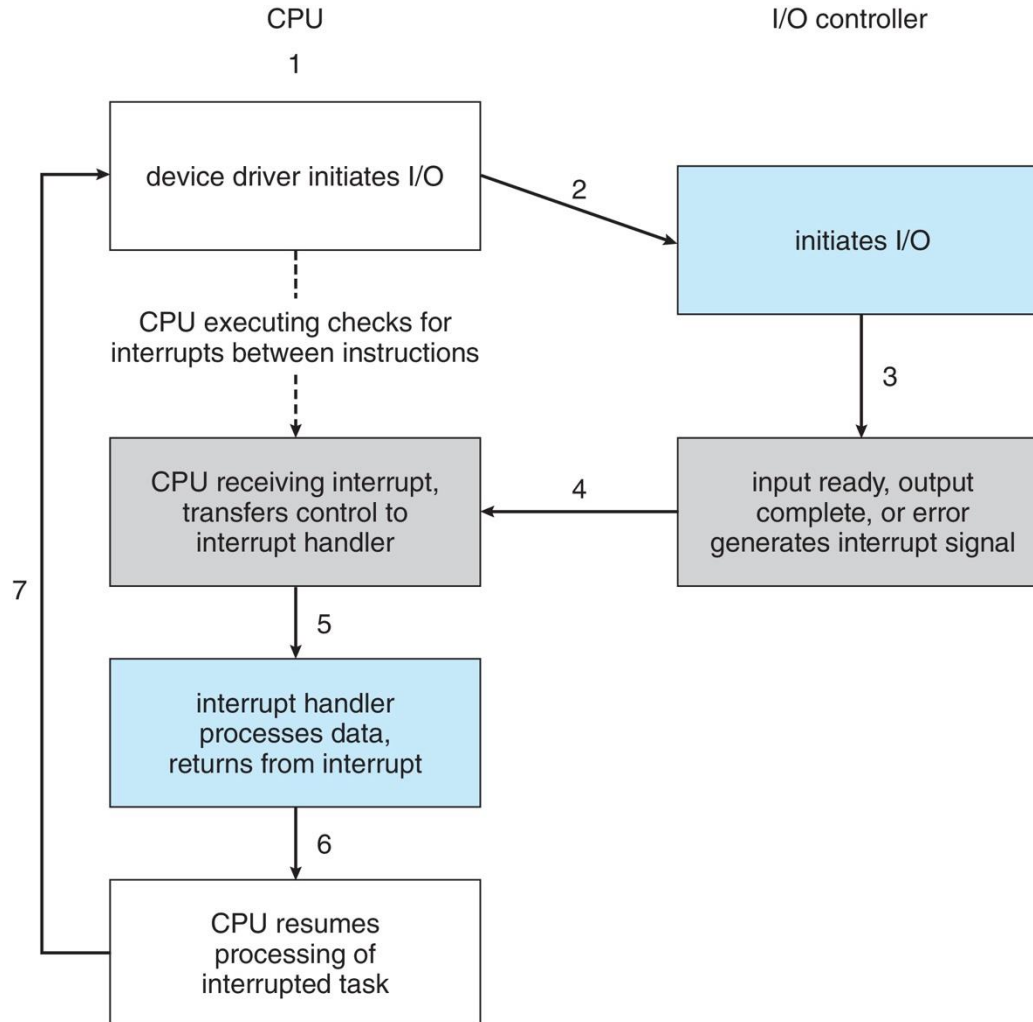
Polling

- For each byte of I/O
 1. Read busy bit from status register until 0
 2. Host sets write bit and write copies data into data-out register
 3. Host sets command-ready bit
 4. Controller sets busy bit, executes transfer
 5. Controller clears busy bit, error bit, command-ready bit when transfer done
- **busy-wait** cycle to wait for I/O from device
 - Reasonable if device is fast
 - But inefficient if device slow
 - CPU switches to other tasks?
 - ▶ But if miss a cycle data overwritten / lost





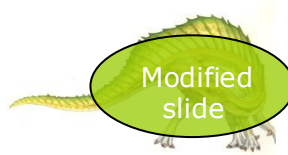
Interrupt-Driven I/O Cycle





Interrupts

- CPU **Interrupt-request line** triggered by I/O device
 - Checked by processor after each instruction
- **Interrupt handler** receives interrupts
 - **Maskable** to ignore or delay some interrupts
- **Interrupt vector** to dispatch interrupt to correct handler





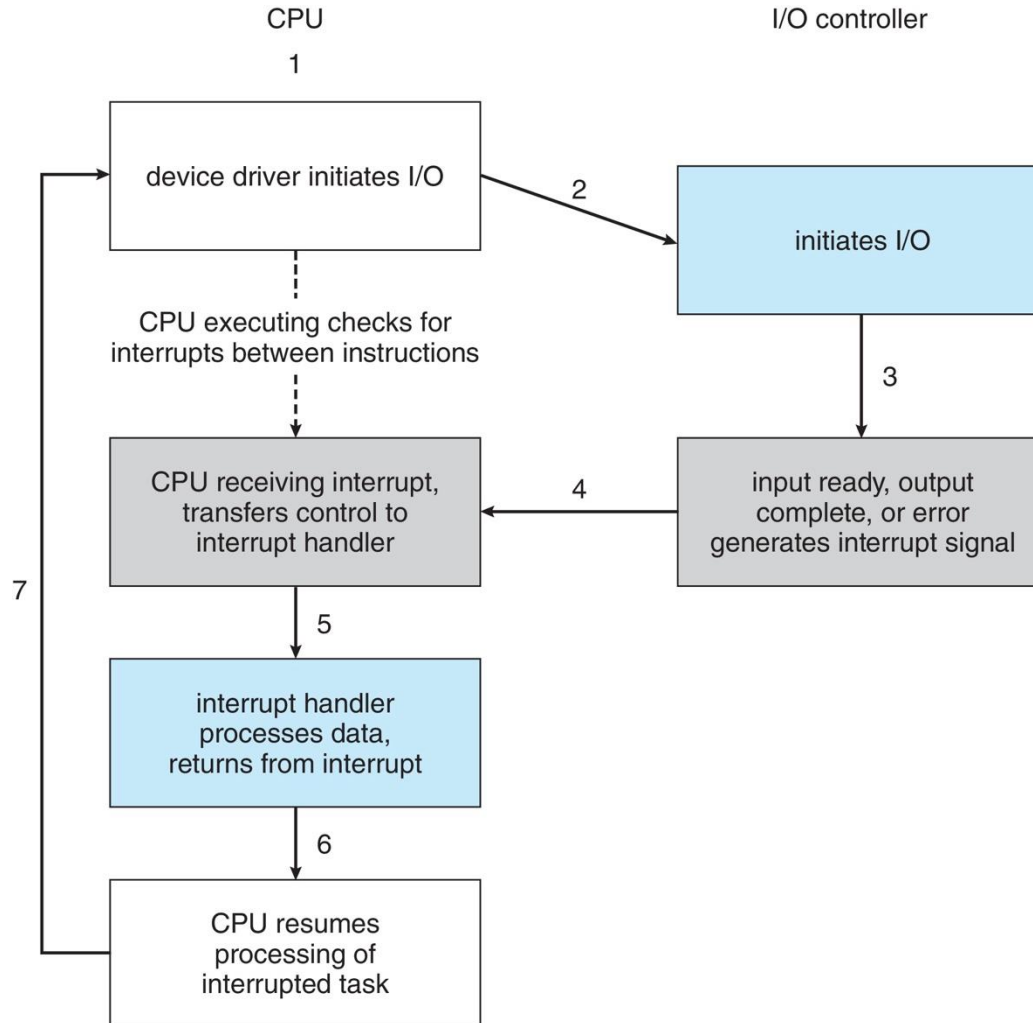
Intel Pentium Processor Event-Vector Table

vector number	description
0	divide error
1	debug exception
2	null interrupt
3	breakpoint
4	INTO-detected overflow
5	bound range exception
6	invalid opcode
7	device not available
8	double fault
9	coprocessor segment overrun (reserved)
10	invalid task state segment
11	segment not present
12	stack fault
13	general protection
14	page fault
15	(Intel reserved, do not use)
16	floating-point error
17	alignment check
18	machine check
19–31	(Intel reserved, do not use)
32–255	maskable interrupts





Interrupt-Driven I/O Cycle

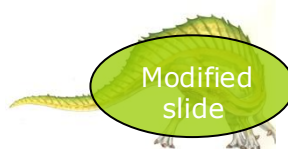


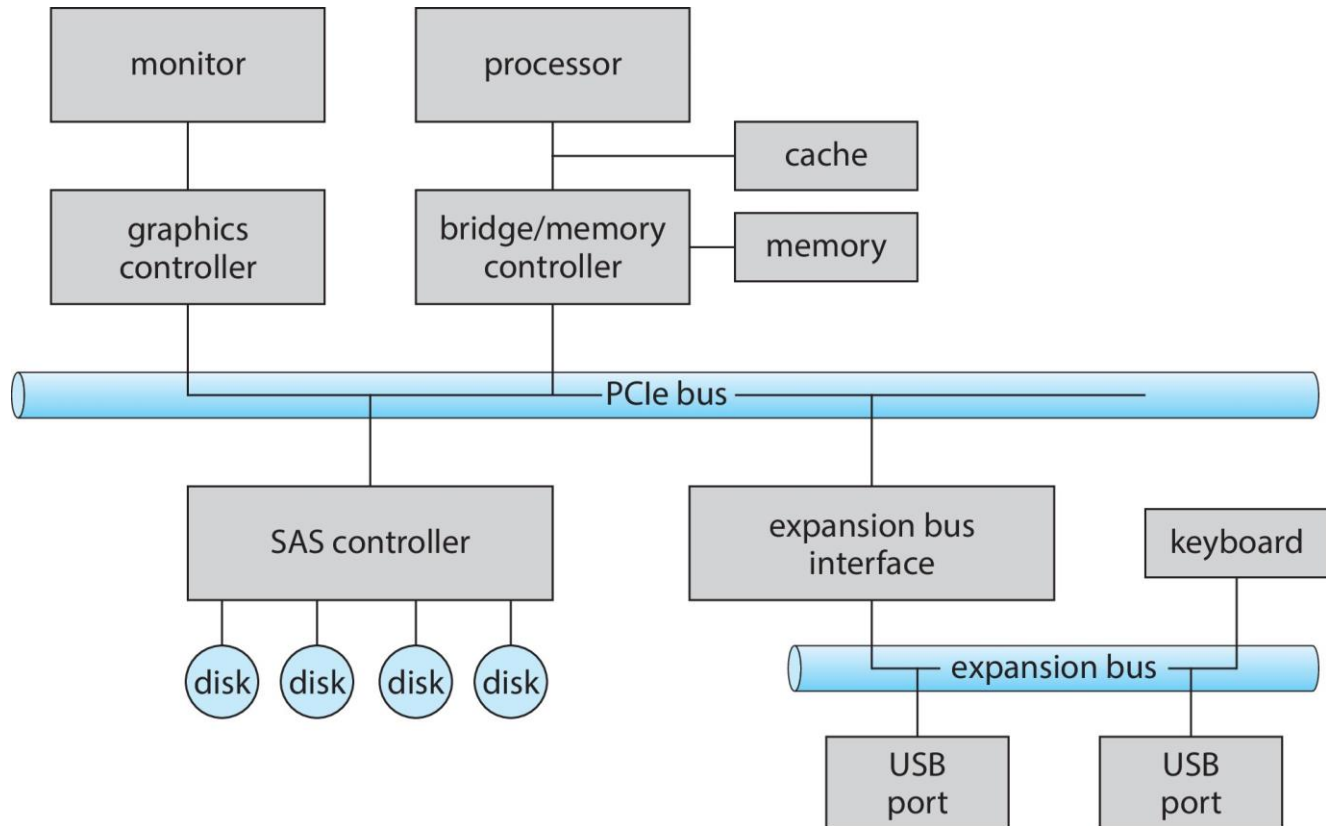


Direct Memory Access

- Used to avoid **programmed I/O** (one byte at a time) for large data movement
- Requires **DMA** controller
- Bypasses CPU to transfer data directly between I/O device and memory
- OS writes DMA command block into memory
 - Source and destination addresses
 - Read or write mode
 - Count of bytes
 - Writes location of command block to DMA controller
 - When done, **interrupts** to signal completion

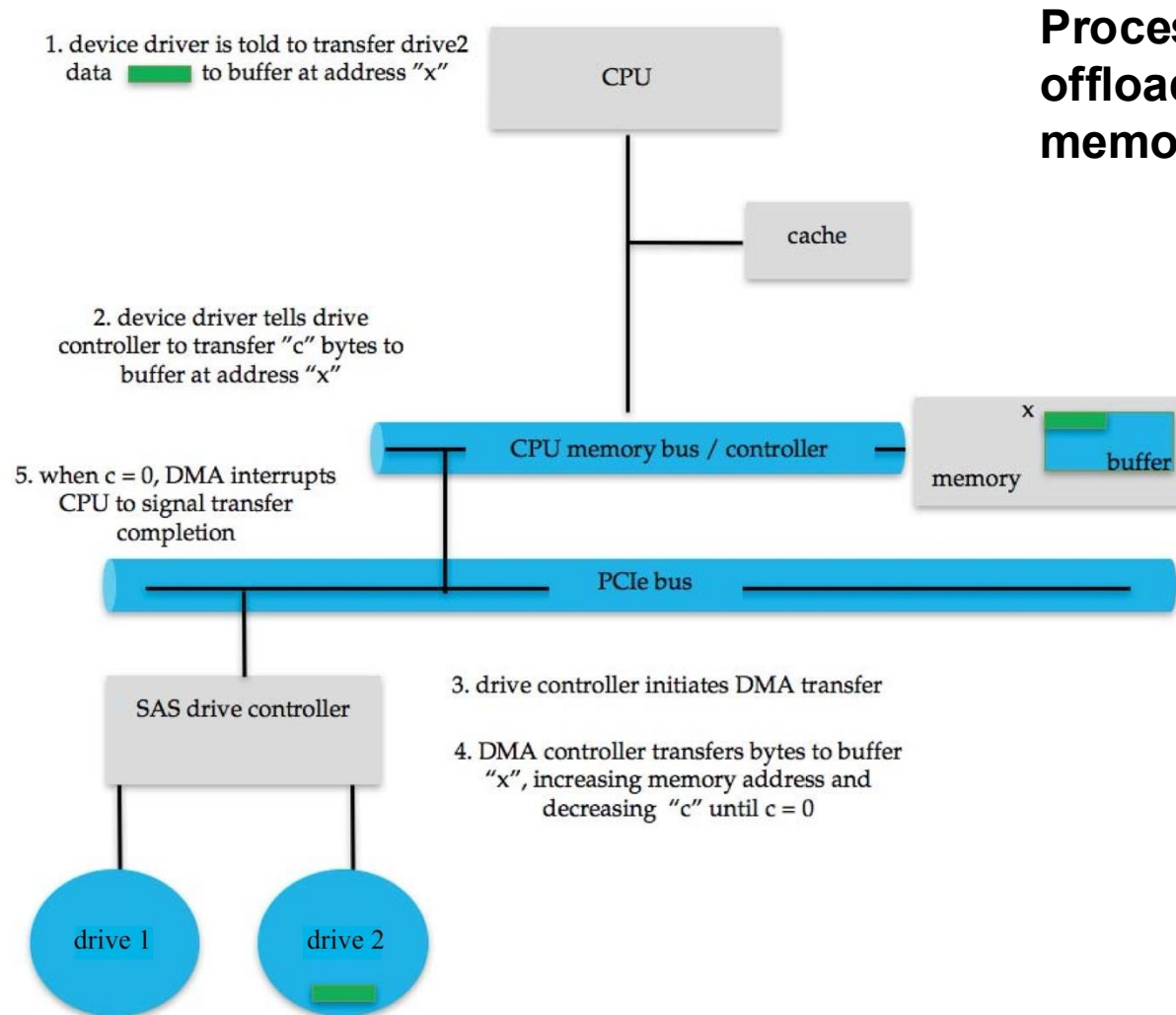
“**Direct memory access (DMA)** is a feature of computer systems that allows certain hardware subsystems to access main system memory independently of the central processing unit (CPU).”



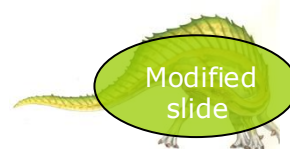




Six Step Process to Perform DMA Transfer



Processor uses DMA to offload *expensive* memory operations





Done!

Done!





